

Daily Assignment on FETCH API

Approach:

Fetch API was used to retrieve blog posts and todo data from JSONPlaceholder APIs. The data was displayed dynamically using DOM manipulation. The Module Pattern was applied to organize the code and improve maintainability.

Code Implementation:

The application uses `fetch()` with `async/await` for API calls. Separate functions handle posts and todos. Error handling is implemented to display appropriate messages when API requests fail.

Challenges Faced:

Managing asynchronous API calls and handling errors correctly were the main challenges. Structuring the code using a design pattern while keeping it simple was also addressed.

Learning Reflection:

This assignment improved my understanding of Fetch API, JSON handling, DOM manipulation, and JavaScript code organization using design patterns.

Solution:

Index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Fetch API Assignment</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <h1>Simple Blogging Platform</h1>
      <p>Fetch API Demo – Blog Posts & Todo List</p>
    </header>
```

```

<main>
  <!-- Blog Section -->
  <section>
    <h2>Latest Blog Posts</h2>
    <p id="postError" class="error"></p>
    <div id="postContainer"></div>
  </section>

  <aside class="todo-box">
    <h2>My Todo List</h2>
    <p id="todoError" class="error"></p>
    <ul id="todoList"></ul>
  </aside>
</main>

<footer>© Fetch API Assignment – Wipro NGA MERN</footer>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background-color: #f4f6f9;
  color: #333;
}

header {
  background: #2c3e50;
  color: white;
  padding: 20px;
  text-align: center;
}

main {
  display: grid;
  grid-template-columns: 2.5fr 1fr;
  gap: 10px;
  padding: 30px;
  max-width: 1200px;
  margin: auto;
}

```

```
}

h2 {
  margin-top: 0;
  border-bottom: 2px solid #eee;
  padding-bottom: 10px;
}

/* Blog Posts */
.post {
  background: white;
  padding: 20px;
  border-radius: 8px;
  margin-bottom: 20px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.05);
}

.post h3 {
  margin-top: 0;
  margin-bottom: 10px;
  color: #2c3e50;
}

.post p {
  line-height: 1.6;
  color: #555;
}

/* Todo Section */
.todo-box {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 4px 10px rgba(0,0,0,0.05);
  position: sticky;
  top: 20px;
}

ul {
  padding-left: 20px;
  margin: 0;
}

li {
  margin-bottom: 8px;
}

.completed {
  text-decoration: line-through;
  color: #999;
}
```

```

.error {
  color: red;
  font-weight: bold;
  margin-bottom: 10px;
}

footer {
  text-align: center;
  padding: 15px;
  font-size: 14px;
  color: #777;
}

```

Script.js

```

const AppModule = (function () {

  const POSTS_URL = "https://jsonplaceholder.typicode.com/posts";
  const TODOS_URL = "https://jsonplaceholder.typicode.com/todos";

  async function fetchData(url) {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error("Fetch failed");
    }
    return response.json();
  }

  async function loadPosts() {
    const postContainer = document.getElementById("postContainer");
    const postError = document.getElementById("postError");

    try {
      const posts = await fetchData(POSTS_URL);
      posts.slice(0, 5).forEach(post => {
        const div = document.createElement("div");
        div.className = "post";
        div.innerHTML = `<h3>${post.title}</h3><p>${post.body}</p>`;
        postContainer.appendChild(div);
      });
    } catch (error) {
      postError.textContent = "Unable to load blog posts.";
    }
  }

  async function loadTodos() {
    const todoList = document.getElementById("todoList");
    const todoError = document.getElementById("todoError");
  }
}

```

```
    try {
      const todos = await fetchData(TODOS_URL);
      todos.slice(0, 10).forEach(todo => {
        const li = document.createElement("li");
        li.textContent = todo.title;
        if (todo.completed) {
          li.classList.add("completed");
        }
        todoList.appendChild(li);
      });
    } catch (error) {
      todoError.textContent = "Unable to load todo list.";
    }
  }

  return {
    init: function () {
      loadPosts();
      loadTodos();
    }
  };
}());

AppModule.init();
```

Snapshot of web page:

